Question 1: Word Segmentation and POS Tagging

What you have to do

You will be given a sentence with all the spaces removed - just one long string of letters. Your program has to do two things, in order:

1. Break the string into the correct words (Segmentation).
2. Label each word with its correct grammar category — noun, verb, adjective, etc. (POS tagging).

You will do this for two languages: English and one morphologically richer language (Spanish or German — any one). The point is to see how well the same technique works on a language where word endings carry a lot of grammatical meaning (like gender and number agreement).

Example (English)

Input:

```
thequickbrownfoxjumpsoverthelazydog
```

Output:

```
[(the, DT), (quick, JJ), (brown, JJ), (fox, NN),
 (jumps, VBZ), (over, IN), (the, DT), (lazy, JJ), (dog, NN)]
```

Example (Spanish)

Input:

```
lacasarojaesgrande
```

Output:

```
[(la, DET), (casa, NOUN), (roja, ADJ), (es, AUX),
 (grande, ADJ)]
```

Notice roja (red) and casa (house) agree in gender — this is something English doesn't have, and it's exactly what makes this comparison interesting.

Step-by-Step Breakdown

Part 1 — Word Segmentation

- Train a trigram language model on your corpus. This model learns which sequences of words are common, so it can guess where one word ends and the next begins.
- Use a dynamic programming algorithm (like Viterbi) to try all reasonable ways of splitting the string and pick the most probable one.

Think of it like this: the algorithm doesn't guess randomly — it scores every possible split

using the language model, and dynamic programming lets it do this efficiently instead of checking every combination one by one.

Part 2 — POS Tagging

- Train a second model that learns two things:
  - Emission probability — how likely a word is to have a certain tag (e.g., “dog” is usually a noun).
  - Transition probability — how likely a tag is to follow the previous two tags (e.g., an adjective is often followed by a noun).
- Use the same kind of dynamic programming approach to tag the segmented words.

Part 3 — Morphology-Aware Tagging

Plain POS tags (like NOUN, ADJ) don’t capture things like gender and number. Add this extra layer:

- Extend your tags to include gender/number information where relevant, e.g., NOUN-Fem Sg, ADJ-Masc-Pl.
- This means your model now has to learn agreement patterns — e.g., a feminine noun is more likely to be followed by a feminine adjective.
- Why this matters: it lets you check something more interesting than “did I get the word right” — you can check “did the model understand grammatical agreement,” which is where English and Spanish/German really differ.

Part 4 — Baseline Comparison

Before judging your trigram+DP models, build two much simpler baselines:

- Segmentation baseline: greedy longest-match - always try to match the longest word from your vocabulary at each point.
- Tagging baseline: most-frequent-tag - just always assign the tag each word most com monly has in training data.

Report your model’s improvement over these baselines. This shows whether your model is actually worth the extra work - a key thing any real NLP evaluation should show.

Part 5 - Error Analysis with Confusion Matrix

Instead of just reporting one accuracy number, also build a confusion matrix for POS tagging - a small table showing which tags get mixed up with which (e.g., is ADJ often mistaken for NOUN?). Also check: how many tagging errors were actually caused by an earlier segmen tation mistake? If a word was split wrong, its tag is almost certainly going to be wrong too - separating these two error sources gives a much clearer picture.

Small Example:

Consider the input:

```
thequickbrownfox
```

The correct segmentation and POS tags are:


```
the/DET quick/ADJ brown/ADJ fox/NOUN
```

Suppose the segmentation model predicts:

```
thequick | brown | fox
```

instead of:

```
the | quick | brown | fox
```

If thequick is then assigned a wrong POS tag, this is counted as an error caused by a segmen tation mistake, because the incorrect word boundary caused the later tagging error.

Now suppose the segmentation is correct:

```
the | quick | brown | fox
```

but the POS tagger predicts: quick → NOUN

while the correct tag is: quick → ADJ

This is a genuine POS tagging error, because the word was segmented correctly but assigned the wrong tag.

For example, a small confusion matrix could be:

Predicted \ Actual	ADJ	NOUN
ADJ	60	15
NOUN	10	80

Here, the value 15 means that 15 words whose correct tag was ADJ were incorrectly predicted as NOUN. Thus, the confusion matrix shows which tags are confused, while the error-source analysis separates errors caused by segmentation from genuine tagging errors.

Corpora to Use

- English: Brown Corpus (via NLTK)
- Spanish: UD Spanish-GSD

```
git clone https://github.com/UniversalDependencies/UD_Spanish-GSD.git
```

• German: UD German-GSD

```
git clone https://github.com/UniversalDependencies/UD_German-GSD.git
```

Use the train split for training, dev split for tuning, and test split for final evaluation. For English, split Brown 80/20 yourself.

What to Submit

1. Code for segmentation model (both languages)
2. Code for POS tagging model, including the morphology-aware tags (both languages)
3. Code for the two baselines

4. Evaluation code: accuracy, confusion matrix, and error-source breakdown
5. Output examples on the sample test strings below, plus a few of your own
6. A short comparative report answering:
    - Where did English and the other language differ most in accuracy?
    - Did agreement-aware tagging actually help, or add noise?
    - How much of the tagging error came from segmentation mistakes vs. genuine tagging mistakes?
    - How much better were your models than the simple baselines?

Sample Test Strings

Spanish

- mispadrespuedenviajar → [(mis, DET), (padres, NOUN), (pueden, VERB), (viajar, VERB)]
- elcielodespejadoesazul → [(el, DET), (cielo, NOUN), (despejado, ADJ), (es, AUX), (azul, ADJ)]

English

- thequickbrownfoxjumpsoverthelazydog → [(the, DT), (quick, JJ), (brown, JJ), (fox, NN), (jumps, VBZ), (over, IN), (the, DT), (lazy, JJ), (dog, NN)]

German

- autobahnmeistereiverwaltungsgebaeude → [(Autobahnmeisterei, NN), (Verwaltungsgebaeude, NN)]

Or potentially:

```
[(Autobahn, NN), (Meisterei, NN), (Verwaltungsgebaeude, NN)]
```

depending on lexicon and probabilities.

Marking Split (out of 100)

Component	Marks
Train/test split & data handling	10
Segmentation model (trigram + DP)	15
POS tagging model (emission + transition)	15
Morphology-aware tagging extension	15
Baseline models + comparison	13
Evaluation (accuracy + confusion matrix + error-source breakdown)	15
Comparative analysis report	17

Question 2 : Implement and Train a Transition-Based Dependency Parser

The goal of this assignment is to build a simple, data-driven dependency parser from scratch. You will implement a transition-based parser that uses a classifier trained on a treebank to make its decisions.

Dataset: You will use the Universal Dependencies English-EWT corpus. You should use the en_ewt-ud-train.conllu file for training your model and the en_ewt-ud-dev.conllu file for evaluat ing its performance.

You can download the data by cloning the official GitHub repository: https://github.com/UniversalDependencies/UD_English-EWT.git

Transition System: You are required to implement a dependency parser based on the arc standard transition system. The system uses a stack, a buffer, and a set of arcs to build a parse tree. The possible transitions are:

1. **SHIFT:** Move the first word from the buffer to the top of the stack.
2. **LEFT-ARC(label):** The word at the top of the stack becomes the head of the second word on the stack. The second word is then popped from the stack.
3. **RIGHT-ARC(label):** The second word on the stack becomes the head of the word at the top of the stack. The top word is then popped from the stack.

Implementation Details:

Your implementation should be divided into the following parts:

Part 1: Data Processing and Oracle Simulation

1. **CoNLL-U Parser:** Write a function to read the .conllu files. It should parse each sentence and store its words, POS tags, and gold-standard head-dependent relationships.
2. **Oracle Simulator:** Write a function that takes a gold-standard parsed sentence (from the CoNLL-U file) and simulates the parsing process. For each step (configuration), it should determine the correct oracle transition (SHIFT, LEFT-ARC, or RIGHT-ARC) that leads to the gold-standard tree. This function will generate your training data: a list of

(configuration, correct_transition) pairs.

Part 2: Feature Extraction and Model Training

1. **Feature Extractor:** Write a function that takes a parser configuration (stack, buffer) and extracts features. For this assignment, you should implement the following simple features:
    - POS tag of the word on top of the stack.
    - POS tag of the second word on the stack (if it exists).
    - POS tag of the first word in the buffer (if it exists).
    - POS tag of the second word in the buffer (if it exists).
2. **Model Training:** Use the training data generated in Part 1 to train a classifier. This classifier will act as your oracle to predict the next transition for a given configuration.

Part 3: Parser Implementation and Evaluation

1. **Parser:** Implement the main parser function. It should take a sentence (a list of words and their POS tags) as input. It will initialize a configuration and then loop, at each step doing the following:
    - Extract features from the current configuration.
    - Use your trained classifier to predict the next transition.
    - Apply the predicted transition to update the configuration. The loop continues until the parsing is complete. The function should return the set of predicted dependency arcs.
2. **Evaluation:** Write an evaluation script that runs your trained parser on the en_ewt-ud dev.conllu dataset. You should calculate the Labeled Attachment Score (LAS), which is the percentage of words that were assigned the correct head and the correct dependency label.

Some example sentences to test your system:

- “The cat sat on the mat.”
- “She eats a green salad.”
- “I saw the man with a telescope.”

Marking Scheme: (40)

1. Part 1: Data Processing and Oracle Simulation (12 marks)
    - 5 marks - Correctly parsing the CoNLL-U file and storing the data structures.
    - 7 marks - Correct implementation of the oracle simulator to generate training in stances.
2. Part 2: Feature Extraction and Model Training (10 marks)

- 5 marks - Implementation of the feature extraction function.
- 5 marks - Correctly training a scikit-learn classifier with the generated features and labels.

3. Part 3: Parser and Evaluation (15 marks)
    - 10 marks - Implementation of the core parsing loop that uses the trained model.
    - 5 marks - Correct implementation of the LAS evaluation metric.
4. Code Quality and Report (3 marks)
    - 3 marks - Well-structured, commented code and a brief report explaining your design choices and final LAS score.

Question 3: Building, Benchmarking, and Deploying an Efficient Spelling Corrector

The goal of this assignment is to build a robust spelling corrector that can handle both non-word and real-word errors within an edit distance of 1. You will implement and compare two different methods for generating candidate corrections, analyze their impact on performance through a “Speed Demon” benchmark, and finally deploy your model as a live interactive application.

Dataset: You will use the Brown Corpus from the NLTK library to build your vocabulary and language model. This corpus is well-balanced and suitable for gathering word frequency and contextual probabilities.

Implementation Details: Your task is to create a spelling corrector by implementing the fol lowing components.

Part 1: Corpus and Model Preparation

1. **Vocabulary and Frequencies:** Process the Brown corpus to create a vocabulary of unique words and a frequency distribution (unigram model).
2. **Language Model:** Create a bigram probability model from the corpus. This will be used to handle real-word errors by considering the context of a word.

Part 2: Candidate Generation Methods

You must implement two distinct methods for generating candidate corrections for a given misspelled word.

1. **Method A: Standard Edit Distance 1 Generation**
    - Write a function that takes a word and generates a set of all possible words at an edit

distance of 1 (deletions, transpositions, replacements, and insertions).

2. **Method B: Symmetric Delete Spelling Correction**

   • This is a highly efficient method for finding candidates.

   • **Preprocessing Step:** Create a dictionary that maps every possible one-character deletion of a vocabulary word back to the original word (e.g., 'ello': ['hello'], 'hllo': ['hello'], ...).

   • **Candidate Generation Step:** To find candidates for a misspelled word, first gen erate all its one-character deletions. Then, look up these deleted variants in your pre-processed dictionary to find matching vocabulary words.

Part 3: Spelling Correction Logic

1. **Non-Word Error Correction:** For a word not found in your vocabulary, use both Method A and Method B to generate candidate sets. The best correction is the candidate with the highest frequency (unigram probability).

2. **Real-Word Error Correction:** For a word that is in the vocabulary but may be incorrect in its context (e.g., "I ate an apple" vs. "I ate an apply"), you must:

   • Generate candidate corrections for the target word using your candidate generation methods.

   • Use your bigram model to calculate the probability of the original phrase (e.g., P(an apple)) versus the probability of phrases with the corrected candidates (e.g., P(an apply)).

   • If a candidate phrase has a significantly higher probability, suggest it as the correction.

Part 4: Evaluation and "Speed Demon" Benchmark

1. **Test Set Generation:** Create a test set by taking 10% of the sentences from the Brown corpus. For each sentence, randomly select one word and introduce a single-edit spelling mistake to create both a non-word and a real-word error version.

2. **Accuracy:** Report the accuracy of your spelling corrector on both the non-word and real word test sets.

3. **Speed Demon Benchmark:** Isolate your non-word error correction logic. Create a batch of exactly 1,000 misspelled words. Pass this entire batch through Method A, and then pass the exact same batch through Method B. Record and print the total execution time (latency) for each method. Write a brief conclusion analyzing the speed difference and explaining exactly why Method B achieved its specific runtime.

Part 5: Live Interactive Application

To bridge the gap between backend algorithm design and user experience, you must package your spelling corrector into an interactive Continuous Terminal CLI.

- Create a Python script that runs a continuous while loop in your terminal.
- The prompt should ask the user to type a sentence.
- Upon pressing Enter, output the corrected sentence.
- Visually highlight any words that were changed (e.g., using terminal ANSI color codes or wrapping the changed word in **ASTERISKS**).
- Print the execution time (latency) of the correction.
- The application must stop when the user inputs the word exit.

Example sentences to test in your application:

- Non-word error: “I hav a good feeling about this.”
- Non-word error: “This is a test sentnce.”
- Real-word error: “I would like to sea the world.”
- Real-word error: “Please meat me at the station.”

Marking Scheme: (40 Marks)

1. Part 1: Corpus and Model Preparation (6 marks)

   - 3 marks - Correctly building the vocabulary and unigram frequency model.
   - 3 marks - Correct implementation of the bigram probability model.

2. Part 2: Candidate Generation Methods (10 marks)

   - 5 marks - Correct implementation of the standard edit distance 1 generation (Method A).
   - 5 marks - Correct implementation of the symmetric delete method, including the pre processing step (Method B).

3. Part 3: Spelling Correction Logic (8 marks)

   - 4 marks - Implementation for handling non-word errors using unigram probabilities.
   - 4 marks - Implementation for handling real-word errors using the bigram model for context.

4. Part 4: Benchmarking and Evaluation (8 marks)

   - 4 marks - Test set generation code and accuracy calculation for both error types.
   - 4 marks - Code for the 1,000-word speed benchmark and a clear written conclusion analyzing the speed difference.

5. Part 5: Live Interactive Application (8 marks)

   - 8 marks - Successfully implementing the continuous Terminal CLI with correct high

lighting, latency reporting, and user input handling.

Question 4: Building an Integrated Background Editor — Live Segmentation, Spelling Correction, and Constituency-Based Grammar Checking

(This question builds directly on Questions 1 and 3. You are not building three separate demos — the same background-running editor must host all three sub-systems on the same live text stream: the joint segmentation + feature-based POS decoder from Question 1, the spelling cor rector from Question 3, and a new constituency-parsing / language-model grammar checker. Reuse the trained models from Q1 and Q3 (do not retrain them from scratch inside this ques tion), and reuse a single shared trigram language model across all three sub-systems wherever one is needed, rather than training three separate LMs.)

The editor runs as a Streamlit web application while a passage is (simulated to be) typed, raising alerts as it goes, and produces a full structural analysis once the passage is complete.

Dataset / Corpus:

- Use the Penn Treebank sample bundled with NLTK (nltk.corpus.treebank) to train the PCFG grammar.
- Use the Brown Corpus (nltk.corpus.brown) for the English language models used in Q3 and Q4. Reuse the trained English trigram language model from Q1 for segmentation scor ing, and reuse the trained vocabulary, unigram/bigram models, and candidate-generation methods from Q3 for spelling correction. Do not retrain these models inside Q4.

- Reuse, unchanged, the vocabulary + unigram/bigram models and candidate-generation functions (Method A and Method B) built in Question 3, and the trained English trigram LM + feature-based POS classifier + beam decoder built in Question 1. Since Q4 uses English passages, only the English Q1 models are required.
- **Test passages:** randomly sample a paragraph (5–8 contiguous sentences) from a large text library such as nltk.corpus.gutenberg, nltk.corpus.brown, or nltk.corpus.reuters. Pick a different random file/passage each run (seed only for debugging).

Task Breakdown:

Part 1: Background Typing Simulation with Segmentation, Spelling, and Gram mar Alerts

1. Stream the randomly sampled passage word-by-word to simulate live typing (time.sleep() between words). The Streamlit application must also provide a live-typing mode in which text entered by the user is processed incrementally rather than only after the complete passage is submitted.
2. **Simulated fast-typing merges (new):** with a small probability p (e.g., p = 0.08, choose and justify your own value) between any two consecutive words, drop the space between them to produce a single merged token — mimicking a real typist who occasionally fails to hit the spacebar in time. This is what gives Question 1's segmentation model a genuine job to do inside this editor (the rest of the passage is already space-delimited, so segmentation would otherwise be a no-op).

As each token arrives, run it through the following live checks, in order, and print a distinctly labeled alert for each that fires:

- **[SEGMENT-ALERT]** — if the token is not found in the vocabulary as a single word (or is unusually long), run Question 1's trained English joint beam-search decoder, restricted to just that token, using the same maximum word length, α, β, and beam-width settings selected during Q1, to check whether it splits into two or more valid words with a better combined score than treating it as one word. If so, replace it with the split words + their predicted POS tags and alert.
- **[SPELL-ALERT]** — for any token still not in the vocabulary after the segmentation check, run Question 3's candidate generation (Method A and/or Method B — your choice, but justify it) and suggest the highest-unigram-frequency candidate as a non-word-error cor rection.
- **[GRAMMAR-ALERT]** — exactly as in the original design: at a fixed word-count trigger interval N (choose and justify N, considering the added cost of the two checks above), re-check the accumulated window of the last N words using bigram/trigram perplexity from the shared LM and flag it if it looks implausible.

4. Real-word errors (Q3-style: an in-vocabulary word that's wrong in context) are checked at the same trigger interval as the grammar check, by comparing the bigram probability of the actual local phrase against nearby edit-distance-1 candidates, and flagged as part of the [GRAMMAR-ALERT] output if a candidate scores significantly higher.
5. Report, separately, the average latency (ms) of: (a) the per-token segmentation+spelling

check, and (b) the per-trigger grammar/real-word check. Confirm both are fast enough not to visibly lag the simulated typing speed.

Part 2: PCFG Constituency Parser

1. Train a PCFG from the Penn Treebank sample (nltk.induce_pcfg or your own rule-probability estimation).

2. Implement a Viterbi/CKY-based most-probable-parse function. Sentences that fail to parse should be flagged as unparseable, not crash the pipeline.

3. **Tagset reconciliation (new):** the POS tags attached to words in Part 1 come from Ques tion 1's English feature-based classifier trained using the Brown Corpus while the PCFG's lexical productions are built over Penn Treebank tags. Before parsing, you must reconcile the two tagsets — either map Q1's output tags onto the closest Penn Treebank tag with a small lookup table, or retag consistently at training time. Document whichever approach you take and any accuracy loss it introduces.

Part 3: Shared Smoothed N-gram Language Model

1. Train one bigram model and one trigram model on the Brown Corpus with add-k smooth ing. These models are used by Q4 for live grammar alerting and final sentence-level analysis. Reuse the trained English trigram LM from Q1 for segmentation and the trained Q3 bigram model for real-word spelling correction; do not retrain those Q1/Q3 models during Q4 execution.

2. For each sentence extracted at the end of the passage, compute its sentence log-probability/perplexity under both models.

Part 4: Final Passage Analysis — Method Comparison

1. At the end of the passage, split it into sentences (using the final, already segmentation corrected and spelling-corrected token stream) and score each sentence with: (a) PCFG parse log-probability (or "unparseable"), (b) bigram log-probability, (c) trigram log-probability.

2. Apply a documented decision rule (e.g., prefer PCFG when it parses and isn't a probability outlier; else trigram if it has adequate coverage; else bigram) to choose the most suitable method per sentence, and give a final grammaticality verdict.

3. Produce a final per-sentence summary table with columns: sentence text, PCFG result, bigram score, trigram score, chosen method, final verdict, number of segmentation merges resolved in this sentence, and number of spelling corrections applied in this sentence.

Part 5: Live Interactive Deployment and Speed Demon Benchmark

1. **Live Deployment:** Package the fully integrated pipeline (segmentation + spelling + grammar checking) as a Streamlit web application. The application must support live typing: as the user types text, the system should process the incoming text incrementally and display live-style alerts for segmentation, spelling, and grammar issues. After the user

finishes entering the passage, the application should display the final PCFG/n-gram sentence anal ysis and total latency.

2. **Speed Demon Benchmark:** construct a batch of exactly 1,000 simulated words (reuse your Part 4 Q3 corrupted-word generator or similar) and pass them through the full per token live-check pipeline (segmentation-check + spelling-check). Record total and aver age per-word latency. Then run the same batch through only the grammar-trigger check in isolation. Report both numbers and write a short conclusion isolating how much latency the segmentation+spelling layer adds on top of the grammar layer, and why.

Comparative Analysis & Report:

- How often did the real-time (word/trigger-level) alerts agree with the final end-of-passage verdict for the same sentence? Discuss disagreements in either direction.
- Compare PCFG-based judgments against bigram/trigram judgments: which caught differ ent error classes (structural vs. local word-sequence)?
- Discuss how your chosen trigger interval and merge probability p affect the false-alert rate versus how quickly errors are caught.
- **New:** Discuss interaction effects between sub-systems — e.g., cases where a spelling correction or a segmentation split changed a sentence enough to flip whether the PCFG could parse it, or changed which method the Part 4 decision rule selected.
- Report the Speed Demon results and comment on whether the added segmentation/spelling layer is cheap enough to keep running live, or whether it should be throttled to the same trigger interval as the grammar check.

Submission Requirements:

- Code for the simulated live-typing background checker, including the merged-token gener ator and all three alert types.

- Code integrating Question 1's trained English beam-search segmentation/POS decoder and Question 3's spelling corrector as callable components (not reimplemented). Code for PCFG training/parsing and the tagset-reconciliation step.
- Code for the shared bigram/trigram LM with add-k smoothing.
- Code for the end-of-passage comparison table (Part 4) and the Part 5 deployment + Speed Demon benchmark.
- A short report covering: chosen trigger interval N and merge probability p and why, cho sen k and why, the tagset-reconciliation approach, the method-selection decision rule, the Speed Demon results, and the comparative analysis above — illustrated with at least two full sample runs on different randomly sampled passages, plus a screenshot/transcript of the live deployment (Part 5) in action.

Marking Scheme (Total 55 Marks):

- **6 Marks:** Correct integration — reusing Q1's trained English decoder and Q3's corrector as-is, with the appropriate Q1/Q3 language models reused and Q4's grammar language models used for sentence-level analysis.

- **10 Marks:** Part 1 — simulated typing with merged-token generation, segmentation alerts, spelling alerts, and trigger-based grammar/real-word alerts, with justified N and p.
- **12 Marks:** PCFG Constituency Parser
  - 5 marks — Correct PCFG induction from Penn Treebank sample.
  - 5 marks — Correct Viterbi/CKY-based most-probable-parse implementation, including graceful failure handling.
  - 2 marks — Correct and clearly documented tagset reconciliation between Q1's tags and the PCFG's tags.
- **8 Marks:** Shared Smoothed N-gram Language Model (4 bigram, 4 trigram), correctly reused across sub-systems.
- **6 Marks:** End-of-passage per-sentence comparison table (including segmentation/spelling columns) and justified decision rule.
- **8 Marks:** Part 5 — Streamlit live-typing deployment and Speed Demon benchmark, in cluding incremental processing, live alerts, final analysis, and latency evaluation.
- **5 Marks:** Comparative Analysis Report, including the new sub-system interaction discus sion.